Docker

Résumé du chapitre : gérer les conteneurs, construire et optimiser des images

1. Introduction

Docker est une plateforme pour **construire des images** et **exécuter des conteneurs**. Ses commandes sont très proches de celles de Podman.

Opération	Docker	Podman
Lancer un conteneur	docker run	podman run
Lister les conteneurs	docker ps	podman ps
Arrêter un conteneur	docker stop	podman stop
Construire une image	docker build	podman build
Copier des fichiers	docker cp	podman cp
Réseaux / Volumes	docker network / volume	podman network / volume

À retenir : différence d'architecture : Docker Engine utilise un service central (**dockerd**), alors que Podman peut fonctionner sans démon permanent.

2. Construire une image avec un Dockerfile

- Fichier par défaut : **Dockerfile** (Docker) ou **Containerfile** (Podman). Les deux outils acceptent les deux noms.
- Commande : `docker build -t myapp .`
- Instructions principales : FROM, RUN, COPY, CMD, ENTRYPOINT...

CMD et ENTRYPOINT

Les deux définissent ce qui s'exécute au démarrage du conteneur :

- **CMD** : commande ou arguments **par défaut**, faciles à remplacer au lancement.
- **ENTRYPOINT** : le **programme principal** du conteneur ; les arguments de `docker run` lui sont transmis.

Situation	Comportement
CMD seul	La commande par défaut peut être remplacée
ENTRYPOINT seul	Programme fixe ; on peut ajouter des arguments
ENTRYPOINT + CMD	Programme fixe avec arguments par défaut modifiables

```
FROM alpine:3.22
ENTRYPOINT ["echo"]
CMD ["Hello Docker"]

# docker run --rm echo-tool           ->  Hello Docker
# docker run --rm echo-tool "Hello"   ->  Hello
```

À retenir : l'association ENTRYPOINT + CMD crée une image qui se comporte comme un outil exécutable avec un argument par défaut.

3. Les layers d'une image

Une image est faite de plusieurs **couches (layers)** qui représentent les modifications du système de fichiers pendant le build.

- **RUN, COPY, ADD** créent normalement des couches de fichiers.
- **CMD, ENV, WORKDIR** modifient la configuration sans forcément créer de couche.

Avantages

- **Réutilisation** : des images peuvent partager les mêmes couches.
- **Économie d'espace** : pas de duplication inutile.
- **Rapidité** : les étapes déjà construites sont réutilisées.
- **Distribution** : seules les couches manquantes sont téléchargées.

Stockage

Élément	Rôle
containerd	Gère notamment les images et les conteneurs
Blobs	Contiennent les données des layers (content store de containerd)
Snapshots	Combinent les layers pour former le système de fichiers du conteneur
OverlayFS	Superpose les couches sous Linux et ajoute une couche inscriptible

```
docker info                          # infos de stockage
docker image inspect custom-nginx:1.0   # détails d'une image
docker image history custom-nginx:1.0   # historique des couches
```

4. Le cache de build

- Docker lit le Dockerfile dans l'ordre et **réutilise** les étapes inchangées (indication `CACHED` avec BuildKit).
- `--no-cache` force la réexécution de toutes les étapes : utile pour diagnostiquer ou éviter un résultat d'installation en cache.
- **BuildKit** est le moteur de construction utilise par Docker.

Optimiser le cache : l'ordre compte

Placer ce qui change **rarement** avant ce qui change **souvent**.

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./   # dépendances d'abord
RUN npm ci                                # reste en cache si inchangé
COPY . .                                  # code source ensuite
CMD ["npm", "start"]
```

Un fichier **.dockerignore** (ex. `.git`, `.env`, `*.log`) réduit le contexte de build et évite d'inclure des fichiers inutiles ou sensibles.

5. Le build multi-stage

Plusieurs étapes dans un même Dockerfile : on **prépare** l'application dans une première étape, puis l'image finale ne garde que ce qui est nécessaire à l'**exécution**.

```
# Étape 1 : dépendances
FROM python:3.12 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Étape 2 : image finale
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
CMD ["python", "app.py"]
```

À retenir : `COPY --from=builder` copie uniquement les fichiers nécessaires depuis une étape précédente vers l'image finale.

6. Optimiser une image Docker : 6 bonnes pratiques

#	Bonne pratique	Détail
1	**Image de base légère**	ex. python:3.12-slim plutot qu'une image complete
2	**Fichier .dockerignore**	exclure .git, .venv, __pycache__, fichiers temporaires, secrets
3	**Ordre des instructions**	dépendances d'abord, code source ensuite
4	**Cache de build**	laisser Docker réutiliser les étapes inchangées
5	**Multi-stage build**	ne garder que le nécessaire à l'exécution
6	**Limiter dépendances et privilèges**	paquets utiles seulement, utilisateur non privilégié

À retenir : l'objectif est une image aussi légère que possible, sans retirer le nécessaire, tout en restant fonctionnelle et maintenable.

7. Questions de compréhension

1. Différence entre une image et un conteneur ?

2. Rôle de FROM ?
3. Différence entre RUN et CMD ?
4. Rôle de ENTRYPOINT ?
5. Différence entre CMD et ENTRYPOINT ?
6. Pourquoi associer ENTRYPOINT et CMD ?
7. Qu'est-ce qu'un layer ?
8. Avantages des layers ?
9. Rôle du cache de build ?
10. Différence entre --no-cache et --pull ?
11. Comment organiser un Dockerfile pour le cache ?
12. À quoi sert .dockerignore ?
13. Comment examiner l'historique d'une image ?
14. Commande pour consulter les logs d'un conteneur ?
15. Pourquoi un stockage persistant pour une base de données ?